

INTERMEDIATE PRACTICE WORKBOOK

SQL Server for ETL & Data Warehousing

203 Hands-On Tasks & Fully Commented Solutions

T-SQL / SQL Server

SalesDB.Sales Schema

27 Topic Sections

Built on the **SalesDB** database — Customers, Employees, Products, Orders and OrdersArchive — covering everything from core SQL fundamentals through joins, window functions, DDL/DML, stored procedures, dimensional modeling, Slowly Changing Dimensions, incremental ETL and the Bronze / Silver / Gold medallion pattern.

How to Use This Workbook

Every task below is written against the **SalesDB.Sales** schema shown here. Each task states a business question in plain English; the solution directly beneath it is a complete, runnable T-SQL statement with comments explaining the non-obvious parts — written to double as interview-reading material.

Work through the sections in order for a structured intermediate-level ramp: fundamentals & joins first, then window functions and data cleaning, then database objects (DDL/DML/views/procedures/indexes), and finally the data-warehousing and ETL sections (dimensional modeling, SCD, incremental loads, data quality and the medallion architecture).

Reference Schema

Table	Key Columns
Sales.Customers	CustomerID (PK), FirstName, LastName, Country, Score
Sales.Employees	EmployeeID (PK), FirstName, LastName, Department, BirthDate, Gender, Salary, ManagerID (self-FK)
Sales.Products	ProductID (PK), Product, Category, Price
Sales.Orders	OrderID (PK), ProductID (FK), CustomerID (FK), SalesPersonID (FK -> Employees), OrderDate, ShipDate, OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime
Sales.OrdersArchive	Same structure as Sales.Orders — historical / archived order records

Note: sections 18-27 introduce a few illustrative warehouse objects (e.g. `dbo.FactSales`, `dbo.DimCustomer`, `Staging.*`, `Bronze/Silver/Gold.*`) that are created as part of those tasks themselves, to practice dimensional modeling and medallion-architecture patterns end to end.

Table of Contents

01	SQL Fundamentals	14 tasks
02	Aggregations	14 tasks
03	Joins	14 tasks
04	Set Operations	6 tasks
05	Subqueries	10 tasks
06	Common Table Expressions (CTEs)	8 tasks
07	Window Functions	16 tasks
08	Data Cleaning	13 tasks
09	DDL — Database Structure	8 tasks
10	DML — Data Manipulation	8 tasks
11	Upsert / MERGE	5 tasks
12	Transactions	5 tasks
13	Temporary Tables	4 tasks
14	Views	5 tasks
15	Stored Procedures	6 tasks
16	Indexes	6 tasks
17	Query Performance	5 tasks
18	Data Warehouse Concepts	5 tasks
19	Dimensional Modeling	5 tasks
20	Surrogate Keys	4 tasks
21	Slowly Changing Dimensions (SCD)	8 tasks
22	ETL — Extract	5 tasks
23	ETL — Transform	6 tasks
24	ETL — Load	5 tasks
25	Incremental ETL	5 tasks
26	Data Quality	8 tasks
27	Bronze / Silver / Gold	5 tasks

Total: 203 tasks & solutions

SQL Fundamentals

1.1 Retrieve every column and every row from the Customers table.

SOLUTION

```
-- Basic full-table read: no filter, no projection
SELECT *
FROM Sales.Customers;
```

1.2 Return only FirstName, LastName and Country for customers located in the USA.

SOLUTION

```
-- Project specific columns and filter with a simple equality predicate
SELECT FirstName, LastName, Country
FROM Sales.Customers
WHERE Country = 'USA';
```

1.3 List the distinct countries that appear in the Customers table.

SOLUTION

```
-- DISTINCT removes duplicate Country values from the result set
SELECT DISTINCT Country
FROM Sales.Customers;
```

1.4 Show all employees ordered from the highest paid to the lowest paid.

SOLUTION

```
-- ORDER BY ... DESC sorts salaries from largest to smallest
SELECT EmployeeID, FirstName, LastName, Salary
FROM Sales.Employees
ORDER BY Salary DESC;
```

1.5 Find customers whose Score is between 300 and 500 (inclusive).

SOLUTION

```
-- BETWEEN is inclusive of both boundary values
SELECT CustomerID, FirstName, LastName, Score
FROM Sales.Customers
WHERE Score BETWEEN 300 AND 500;
```

1.6 Find products that belong to either the 'Bikes' or 'Accessories' category.

SOLUTION

```
-- IN() is shorthand for multiple OR conditions on the same column
SELECT ProductID, Product, Category, Price
FROM Sales.Products
WHERE Category IN ('Bikes', 'Accessories');
```

1.7 Find customers whose last name starts with the letter 'M'.

SOLUTION

```
-- LIKE with a trailing wildcard matches any string starting with 'M'
SELECT CustomerID, FirstName, LastName
FROM Sales.Customers
WHERE LastName LIKE 'M%';
```

1.8 List customers who have not yet received a Score (NULL).

SOLUTION

```
-- NULL can only be tested with IS NULL, never with '= NULL'
SELECT CustomerID, FirstName, LastName
FROM Sales.Customers
WHERE Score IS NULL;
```

1.9 List customers who do have a Score recorded (NOT NULL).

SOLUTION

```
SELECT CustomerID, FirstName, LastName, Score
FROM Sales.Customers
WHERE Score IS NOT NULL;
```

1.10 Find orders where Quantity is greater than 2 AND Sales is greater than 50.

SOLUTION

```
-- AND requires both predicates to be true for the row to be returned
SELECT OrderID, ProductID, Quantity, Sales
FROM Sales.Orders
WHERE Quantity > 2 AND Sales > 50;
```

1.11 Classify every customer's Score into 'Low' (<300), 'Medium' (300-600) or 'High' (>600) using CASE.

SOLUTION

```
-- CASE WHEN implements inline if/else logic inside a SELECT
SELECT
    CustomerID,
    Score,
    CASE
        WHEN Score IS NULL THEN 'Unknown'
        WHEN Score < 300   THEN 'Low'
        WHEN Score <= 600  THEN 'Medium'
        ELSE 'High'
    END AS ScoreBand
FROM Sales.Customers;
```

1.12 Return the 5 highest paid employees.

SOLUTION

```
-- TOP limits the number of rows returned after ordering
SELECT TOP (5) EmployeeID, FirstName, LastName, Salary
FROM Sales.Employees
ORDER BY Salary DESC;
```

1.13 Return orders 11 through 20 when sorted by OrderDate (pagination).

SOLUTION

```
-- OFFSET/FETCH implements page-by-page pagination; ORDER BY is required
SELECT OrderID, CustomerID, OrderDate, Sales
FROM Sales.Orders
ORDER BY OrderDate
OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY;
```

1.14 Return the 3 most recently placed orders with Sales greater than 0.

SOLUTION

```
-- Combine WHERE, ORDER BY and TOP in a single query
SELECT TOP (3) OrderID, CustomerID, OrderDate, Sales
FROM Sales.Orders
WHERE Sales > 0
ORDER BY OrderDate DESC;
```

Aggregations

2.1 Count the total number of customers.

SOLUTION

```
SELECT COUNT(*) AS TotalCustomers
FROM Sales.Customers;
```

2.2 Count how many distinct countries customers come from.

SOLUTION

```
-- COUNT(DISTINCT col) counts unique, non-NULL values only
SELECT COUNT(DISTINCT Country) AS DistinctCountries
FROM Sales.Customers;
```

2.3 Calculate the total Sales amount across all orders.

SOLUTION

```
SELECT SUM(Sales) AS TotalSales
FROM Sales.Orders;
```

2.4 Calculate the average Price of all products.

SOLUTION

```
SELECT AVG(Price) AS AvgPrice
FROM Sales.Products;
```

2.5 Find the minimum and maximum employee salary in a single query.

SOLUTION

```
SELECT MIN(Salary) AS MinSalary, MAX(Salary) AS MaxSalary
FROM Sales.Employees;
```

2.6 Count how many orders each customer has placed.

SOLUTION

```
-- GROUP BY collapses rows into one bucket per CustomerID
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Sales.Orders
GROUP BY CustomerID;
```

2.7 Calculate total Sales generated by each product.

SOLUTION

```
SELECT ProductID, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY ProductID;
```

2.8 Calculate the average customer Score per Country.

SOLUTION

```
SELECT Country, AVG(Score) AS AvgScore
FROM Sales.Customers
GROUP BY Country;
```

2.9 Count how many employees work in each Department.

SOLUTION

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Sales.Employees
GROUP BY Department;
```

2.10 List departments that have more than 2 employees.

SOLUTION

```
-- Having filters groups AFTER aggregation; WHERE cannot reference COUNT()
SELECT Department, COUNT(*) AS EmployeeCount
FROM Sales.Employees
GROUP BY Department
HAVING COUNT(*) > 2;
```

2.11 List customers whose combined order Sales exceed 1000.

SOLUTION

```
SELECT CustomerID, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY CustomerID
HAVING SUM(Sales) > 1000;
```

2.12 Calculate total Quantity sold for each order year.

SOLUTION

```
-- YEAR() extracts the calendar year, used here as the grouping key
SELECT YEAR(OrderDate) AS OrderYear, SUM(Quantity) AS TotalQuantity
FROM Sales.Orders
GROUP BY YEAR(OrderDate);
```

2.13 Show how many orders have status 'Shipped' compared to the overall order count.

SOLUTION

```
-- Conditional aggregation: SUM of a CASE expression acts like a filtered COUNT
SELECT
    COUNT(*) AS TotalOrders,
    SUM(CASE WHEN OrderStatus = 'Shipped' THEN 1 ELSE 0 END) AS ShippedOrders
FROM Sales.Orders;
```

2.14 For each salesperson, show the number of orders, total sales, and average sale amount.

SOLUTION

```
SELECT
    SalesPersonID,
    COUNT(*)      AS OrderCount,
    SUM(Sales)    AS TotalSales,
    AVG(Sales)    AS AvgSaleAmount
FROM Sales.Orders
GROUP BY SalesPersonID;
```

Joins

3.1 List every order together with the customer's first and last name.

SOLUTION

```
-- INNER JOIN only returns rows that match on both sides
SELECT o.OrderID, c.FirstName, c.LastName, o.Sales
FROM Sales.Orders AS o
INNER JOIN Sales.Customers AS c
    ON o.CustomerID = c.CustomerID;
```

3.2 List every order together with the product name that was sold.

SOLUTION

```
SELECT o.OrderID, p.Product, o.Quantity, o.Sales
FROM Sales.Orders AS o
INNER JOIN Sales.Products AS p
    ON o.ProductID = p.ProductID;
```

3.3 Find customers who have never placed an order.

SOLUTION

```
-- LEFT JOIN keeps every customer; unmatched orders come back as NULL
SELECT c.CustomerID, c.FirstName, c.LastName
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o
    ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

3.4 Find products that have never appeared in any order (using RIGHT JOIN).

SOLUTION

```
-- RIGHT JOIN keeps every row from Products, even with no matching Orders row
SELECT p.ProductID, p.Product
FROM Sales.Orders AS o
RIGHT JOIN Sales.Products AS p
    ON o.ProductID = p.ProductID
WHERE o.OrderID IS NULL;
```

3.5 Produce a combined list of customers and orders that keeps unmatched rows from both sides.

SOLUTION

```
-- FULL OUTER JOIN returns matches plus unmatched rows from either table
SELECT c.CustomerID, c.FirstName, o.OrderID, o.Sales
FROM Sales.Customers AS c
FULL OUTER JOIN Sales.Orders AS o
    ON c.CustomerID = o.CustomerID;
```

3.6 Generate every possible combination of the 3 cheapest products with the 3 highest-scoring customers.

SOLUTION

```
-- CROSS JOIN produces the Cartesian product (no ON clause)
SELECT c.CustomerID, c.FirstName, p.Product, p.Price
FROM (SELECT TOP (3) * FROM Sales.Customers ORDER BY Score DESC) AS c
CROSS JOIN (SELECT TOP (3) * FROM Sales.Products ORDER BY Price ASC) AS p;
```

3.7 List every employee together with the name of their manager.

SOLUTION

```
-- SELF JOIN: the same table is joined to itself under two aliases
SELECT
    e.EmployeeID, e.FirstName + ' ' + e.LastName AS Employee,
    m.FirstName + ' ' + m.LastName AS Manager
FROM Sales.Employees AS e
LEFT JOIN Sales.Employees AS m
    ON e.ManagerID = m.EmployeeID;
```


3.8 Build one report row per order showing customer name, product name and the salesperson's name.

SOLUTION

```
-- Chaining several INNER JOINS to denormalize four tables into one row
SELECT
    o.OrderID,
    c.FirstName + ' ' + c.LastName AS Customer,
    p.Product,
    e.FirstName + ' ' + e.LastName AS SalesPerson,
    o.Sales
FROM Sales.Orders AS o
INNER JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID
INNER JOIN Sales.Products AS p ON o.ProductID = p.ProductID
INNER JOIN Sales.Employees AS e ON o.SalesPersonID = e.EmployeeID;
```

3.9 Show total sales per customer, including customers with zero orders (show 0, not NULL).

SOLUTION

```
-- LEFT JOIN + COALESCE turns 'no orders' into an explicit 0 instead of NULL
SELECT
    c.CustomerID,
    c.FirstName,
    COALESCE(SUM(o.Sales), 0) AS TotalSales
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o
    ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FirstName;
```

3.10 Combine live Orders with OrdersArchive, then join to Customers for a full historical view.

SOLUTION

```
-- UNION ALL first merges both order sources, then the result is joined once
SELECT c.FirstName, c.LastName, allo.OrderID, allo.Sales
FROM (
    SELECT OrderID, CustomerID, Sales FROM Sales.Orders
    UNION ALL
    SELECT OrderID, CustomerID, Sales FROM Sales.OrdersArchive
) AS allo
INNER JOIN Sales.Customers AS c
    ON allo.CustomerID = c.CustomerID;
```

3.11 Find the best-selling product (by total Sales) inside each Category.

SOLUTION

```
-- Join products to a per-product sales aggregate, then rank within category
WITH ProductTotals AS (
    SELECT p.ProductID, p.Product, p.Category, SUM(o.Sales) AS TotalSales
    FROM Sales.Products AS p
    INNER JOIN Sales.Orders AS o ON p.ProductID = o.ProductID
    GROUP BY p.ProductID, p.Product, p.Category
)
SELECT *
FROM (
    SELECT *, RANK() OVER (PARTITION BY Category ORDER BY TotalSales DESC) AS rnk
    FROM ProductTotals
) AS ranked
WHERE rnk = 1;
```

3.12 Find employees who currently have no one reporting to them (no direct reports).

SOLUTION

```
-- Anti-join pattern: LEFT JOIN to the 'reports' side, keep only unmatched rows
SELECT mgr.EmployeeID, mgr.FirstName, mgr.LastName
FROM Sales.Employees AS mgr
LEFT JOIN Sales.Employees AS rep
    ON rep.ManagerID = mgr.EmployeeID
WHERE rep.EmployeeID IS NULL;
```

3.13 List employees together with a peer who shares the same manager (excluding themselves).

SOLUTION

```
SELECT e1.FirstName AS Employee, e2.FirstName AS Peer, e1.ManagerID
FROM Sales.Employees AS e1
INNER JOIN Sales.Employees AS e2
    ON e1.ManagerID = e2.ManagerID
    AND e1.EmployeeID <> e2.EmployeeID;
```

3.14 List orders for customers in 'USA' who bought products in the 'Bikes' category.

SOLUTION

```
-- Filtering conditions can live on either joined table in the same WHERE clause
SELECT o.OrderID, c.Country, p.Category, o.Sales
FROM Sales.Orders AS o
INNER JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID
INNER JOIN Sales.Products AS p ON o.ProductID = p.ProductID
WHERE c.Country = 'USA' AND p.Category = 'Bikes';
```

Set Operations

4.1 Combine all rows from Orders and OrdersArchive into one result set, keeping duplicates.

SOLUTION

```
-- UNION ALL keeps every row from both queries, including duplicates (fast)
SELECT OrderID, CustomerID, Sales, 'Current' AS SourceTable FROM Sales.Orders
UNION ALL
SELECT OrderID, CustomerID, Sales, 'Archive' AS SourceTable FROM Sales.OrdersArchive;
```

4.2 List every distinct OrderStatus value that appears in either Orders or OrdersArchive.

SOLUTION

```
-- Plain UNION removes duplicate rows across both queries
SELECT OrderStatus FROM Sales.Orders
UNION
SELECT OrderStatus FROM Sales.OrdersArchive;
```

4.3 Find CustomerIDs that exist in Orders but have never appeared in OrdersArchive.

SOLUTION

```
-- EXCEPT returns rows from the first query that are absent from the second
SELECT CustomerID FROM Sales.Orders
EXCEPT
SELECT CustomerID FROM Sales.OrdersArchive;
```

4.4 Find ProductIDs that were sold in BOTH Orders and OrdersArchive.

SOLUTION

```
-- INTERSECT returns only rows present in both result sets
SELECT ProductID FROM Sales.Orders
INTERSECT
SELECT ProductID FROM Sales.OrdersArchive;
```

4.5 Build one unified order list with a literal column that flags which table each row came from.

SOLUTION

```
SELECT OrderID, CustomerID, OrderDate, Sales, 'Live' AS RecordSource FROM Sales.Orders
UNION ALL
SELECT OrderID, CustomerID, OrderDate, Sales, 'Archived' AS RecordSource FROM Sales.OrdersArchive;
```

4.6 Count the total number of order records across Orders and OrdersArchive combined.

SOLUTION

```
-- Wrap the UNION ALL in a derived table so it can be aggregated
SELECT COUNT(*) AS TotalOrderRecords
FROM (
    SELECT OrderID FROM Sales.Orders
    UNION ALL
    SELECT OrderID FROM Sales.OrdersArchive
) AS combined;
```

Subqueries

5.1 Find customers whose Score is higher than the average Score of all customers.

SOLUTION

```
-- The scalar subquery returns a single value the outer query compares against
SELECT CustomerID, FirstName, Score
FROM Sales.Customers
WHERE Score > (SELECT AVG(Score) FROM Sales.Customers);
```

5.2 Find customers who have placed at least one order.

SOLUTION

```
SELECT CustomerID, FirstName, LastName
FROM Sales.Customers
WHERE CustomerID IN (SELECT CustomerID FROM Sales.Orders);
```

5.3 Find customers who have never placed an order, using NOT EXISTS.

SOLUTION

```
-- NOT EXISTS stops scanning as soon as one matching row is found (efficient)
SELECT c.CustomerID, c.FirstName
FROM Sales.Customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM Sales.Orders AS o WHERE o.CustomerID = c.CustomerID
);
```

5.4 Find orders whose Sales value is above the average Sales for that same product (correlated subquery).

SOLUTION

```
-- The inner query re-evaluates per outer row because it references o1.ProductID
SELECT o1.OrderID, o1.ProductID, o1.Sales
FROM Sales.Orders AS o1
WHERE o1.Sales > (
    SELECT AVG(o2.Sales)
    FROM Sales.Orders AS o2
    WHERE o2.ProductID = o1.ProductID
);
```

5.5 Find customers who have at least one order shipped to an address containing 'Cairo'.

SOLUTION

```
SELECT c.CustomerID, c.FirstName
FROM Sales.Customers AS c
WHERE EXISTS (
    SELECT 1 FROM Sales.Orders AS o
    WHERE o.CustomerID = c.CustomerID AND o.ShipAddress LIKE '%Cairo%'
);
```

5.6 Find products that have never been sold.

SOLUTION

```
SELECT p.ProductID, p.Product
FROM Sales.Products AS p
WHERE NOT EXISTS (
    SELECT 1 FROM Sales.Orders AS o WHERE o.ProductID = p.ProductID
);
```

5.7 Find employees who earn more than their own manager.

SOLUTION

```
-- Correlated subquery looks up the manager's salary for each employee row
SELECT e.EmployeeID, e.FirstName, e.Salary
FROM Sales.Employees AS e
WHERE e.Salary > (
    SELECT m.Salary FROM Sales.Employees AS m WHERE m.EmployeeID = e.ManagerID
);
```

5.8 For every customer, show their name and how many orders they have placed (correlated scalar subquery in SELECT).

SOLUTION

```
SELECT
    c.CustomerID,
    c.FirstName,
    (SELECT COUNT(*) FROM Sales.Orders AS o WHERE o.CustomerID = c.CustomerID) AS OrderCount
FROM Sales.Customers AS c;
```

5.9 Find the 3 products with the highest total sales, using a derived table in the FROM clause.

SOLUTION

```
-- A subquery in FROM (a 'derived table') is aggregated first, then filtered
SELECT TOP (3) *
FROM (
    SELECT ProductID, SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY ProductID
) AS product_totals
ORDER BY TotalSales DESC;
```

5.10 Find the customer(s) who purchased the single most expensive product ever sold.

SOLUTION

```
-- Nested subqueries: innermost finds the max price, next finds that ProductID
SELECT DISTINCT c.CustomerID, c.FirstName
FROM Sales.Customers AS c
INNER JOIN Sales.Orders AS o ON c.CustomerID = o.CustomerID
WHERE o.ProductID = (
    SELECT TOP (1) ProductID
    FROM Sales.Products
    WHERE Price = (SELECT MAX(Price) FROM Sales.Products)
);
```

Common Table Expressions (CTEs)

6.1 Using a CTE, calculate total sales per customer and then keep only those above 500.

SOLUTION

```
-- WITH defines a named, reusable result set for the query that follows it
WITH CustomerTotals AS (
    SELECT CustomerID, SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT * FROM CustomerTotals WHERE TotalSales > 500;
```

6.2 Using two CTEs, show each customer's total sales next to their total order count.

SOLUTION

```
-- Multiple CTEs are separated by commas and can be joined like normal tables
WITH SalesCte AS (
    SELECT CustomerID, SUM(Sales) AS TotalSales FROM Sales.Orders GROUP BY CustomerID
),
OrdersCte AS (
    SELECT CustomerID, COUNT(*) AS OrderCount FROM Sales.Orders GROUP BY CustomerID
)
SELECT s.CustomerID, s.TotalSales, o.OrderCount
FROM SalesCte AS s
INNER JOIN OrdersCte AS o ON s.CustomerID = o.CustomerID;
```

6.3 Using a CTE for average price per category, show each product alongside the category average.

SOLUTION

```
WITH CategoryAvg AS (
    SELECT Category, AVG(Price) AS AvgCategoryPrice
    FROM Sales.Products
    GROUP BY Category
)
SELECT p.Product, p.Category, p.Price, ca.AvgCategoryPrice
FROM Sales.Products AS p
INNER JOIN CategoryAvg AS ca ON p.Category = ca.Category;
```

6.4 Build a recursive CTE that lists every employee together with their reporting level under the top manager.

SOLUTION

```
-- The anchor member starts at the top; the recursive member walks down ManagerID
WITH EmployeeHierarchy AS (
    -- Anchor: employees with no manager are level 1 (the top)
    SELECT EmployeeID, FirstName, ManagerID, 1 AS Level
    FROM Sales.Employees
    WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive member: join each employee to their already-processed manager
    SELECT e.EmployeeID, e.FirstName, e.ManagerID, eh.Level + 1
    FROM Sales.Employees AS e
    INNER JOIN EmployeeHierarchy AS eh ON e.ManagerID = eh.EmployeeID
)
SELECT * FROM EmployeeHierarchy
ORDER BY Level;
```

6.5 Find possible duplicate customer records that share the same FirstName and LastName.

SOLUTION

```
WITH NameCounts AS (  
    SELECT FirstName, LastName, COUNT(*) AS Cnt  
    FROM Sales.Customers  
    GROUP BY FirstName, LastName  
)  
SELECT c.*  
FROM Sales.Customers AS c  
INNER JOIN NameCounts AS n  
    ON c.FirstName = n.FirstName AND c.LastName = n.LastName  
WHERE n.Cnt > 1;
```

6.6 Rank customers by Score within their own Country using a CTE plus a window function.

SOLUTION

```
WITH RankedCustomers AS (  
    SELECT  
        CustomerID, Country, Score,  
        RANK() OVER (PARTITION BY Country ORDER BY Score DESC) AS CountryRank  
    FROM Sales.Customers  
)  
SELECT * FROM RankedCustomers ORDER BY Country, CountryRank;
```

6.7 Stage a cleaned version of Orders (trimmed addresses, uppercase status) using a CTE before the final SELECT.

SOLUTION

```
WITH CleanedOrders AS (  
    SELECT  
        OrderID,  
        CustomerID,  
        UPPER(LTRIM(RTRIM(OrderStatus))) AS OrderStatus,  
        LTRIM(RTRIM(ShipAddress)) AS ShipAddress  
    FROM Sales.Orders  
)  
SELECT * FROM CleanedOrders WHERE OrderStatus = 'SHIPPED';
```

6.8 Chain three CTEs together to mimic a mini bronze -> silver -> gold pipeline in a single query.

SOLUTION

```
-- Each CTE builds on the previous one, mirroring a medallion pipeline  
WITH Bronze AS (  
    SELECT * FROM Sales.Orders          -- raw, unmodified extract  
) ,  
Silver AS (  
    SELECT OrderID, CustomerID, ProductID,  
        UPPER(LTRIM(RTRIM(OrderStatus))) AS OrderStatus,  
        Sales  
    FROM Bronze  
    WHERE Sales IS NOT NULL AND Quantity > 0    -- basic validation  
) ,  
Gold AS (  
    SELECT CustomerID, SUM(Sales) AS TotalSales    -- business-ready aggregate  
    FROM Silver  
    GROUP BY CustomerID  
)  
SELECT * FROM Gold ORDER BY TotalSales DESC;
```

Window Functions

7.1 For each customer, number their orders in the order they were placed and return only the first order.

SOLUTION

```
-- ROW_NUMBER() assigns a unique, increasing number within each partition
WITH Numbered AS (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS rn
  FROM Sales.Orders
)
SELECT * FROM Numbered WHERE rn = 1;
```

7.2 Rank customers by their total sales, allowing tied ranks to share the same number.

SOLUTION

```
WITH CustomerTotals AS (
  SELECT CustomerID, SUM(Sales) AS TotalSales
  FROM Sales.Orders
  GROUP BY CustomerID
)
SELECT *, RANK() OVER (ORDER BY TotalSales DESC) AS SalesRank
FROM CustomerTotals;
```

7.3 Rank products by Price within their Category, without leaving gaps after ties (DENSE_RANK).

SOLUTION

```
SELECT
  ProductID, Product, Category, Price,
  DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS PriceRank
FROM Sales.Products;
```

7.4 For each customer, show the date of their previous order next to the current one.

SOLUTION

```
-- LAG() looks one row back within the partition's ORDER BY sequence
SELECT
  CustomerID, OrderID, OrderDate,
  LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS PreviousOrderDate
FROM Sales.Orders;
```

7.5 For each customer, show the date of their next order next to the current one.

SOLUTION

```
-- LEAD() looks one row forward within the partition's ORDER BY sequence
SELECT
  CustomerID, OrderID, OrderDate,
  LEAD(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS NextOrderDate
FROM Sales.Orders;
```

7.6 Show a running total of Sales per customer, ordered by OrderDate.

SOLUTION

```
-- SUM() OVER with PARTITION BY resets the running total for every customer
SELECT
  CustomerID, OrderID, OrderDate, Sales,
  SUM(Sales) OVER (PARTITION BY CustomerID ORDER BY OrderDate
                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM Sales.Orders;
```


7.7 Show a running total of Sales across the whole company, ordered by OrderDate.

SOLUTION

```
-- No PARTITION BY means the running total accumulates over every row
SELECT
    OrderID, OrderDate, Sales,
    SUM(Sales) OVER (ORDER BY OrderDate
                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS CompanyRunningTotal
FROM Sales.Orders;
```

7.8 Show every product's price next to the average price of its category, without collapsing rows.

SOLUTION

```
-- A window AVG keeps one row per product while still 'seeing' the whole group
SELECT
    ProductID, Product, Category, Price,
    AVG(Price) OVER (PARTITION BY Category) AS CategoryAvgPrice
FROM Sales.Products;
```

7.9 Show every order together with the total number of orders that customer has placed.

SOLUTION

```
SELECT
    OrderID, CustomerID, OrderDate,
    COUNT(*) OVER (PARTITION BY CustomerID) AS CustomerOrderCount
FROM Sales.Orders;
```

7.10 Remove duplicate rows from OrdersArchive, keeping only one copy of each OrderID.

SOLUTION

```
-- Classic dedup pattern: number duplicates, then delete everything but rn = 1
WITH Duplicates AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY OrderID ORDER BY CreationTime DESC) AS rn
    FROM Sales.OrdersArchive
)
DELETE FROM Duplicates WHERE rn > 1;
```

7.11 For each customer, return only their most recent order.

SOLUTION

```
WITH Latest AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC) AS rn
    FROM Sales.Orders
)
SELECT * FROM Latest WHERE rn = 1;
```

7.12 Show each order's Sales as a percentage of total company sales.

SOLUTION

```
-- SUM() OVER() with empty parentheses computes the grand total for every row
SELECT
    OrderID, Sales,
    CAST(Sales * 100.0 / SUM(Sales) OVER () AS DECIMAL(5,2)) AS PctOfTotalSales
FROM Sales.Orders;
```

7.13 Split customers into 4 equally sized groups (quartiles) based on Score.

SOLUTION

```
-- NTILE(4) distributes rows as evenly as possible into 4 numbered buckets
SELECT
    CustomerID, Score,
    NTILE(4) OVER (ORDER BY Score DESC) AS ScoreQuartile
FROM Sales.Customers
WHERE Score IS NOT NULL;
```

7.14 For each customer, compute a rolling average of their last 3 orders' Sales.

SOLUTION

```
-- ROWS BETWEEN 2 PRECEDING AND CURRENT ROW gives a 3-row moving window
SELECT
    CustomerID, OrderID, OrderDate, Sales,
    AVG(Sales) OVER (PARTITION BY CustomerID ORDER BY OrderDate
                     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS Rolling3OrderAvg
FROM Sales.Orders;
```

7.15 Show, for each customer, the very first product they ever ordered.

SOLUTION

```
-- FIRST_VALUE returns the first row's value within the ordered window frame
SELECT DISTINCT
    CustomerID,
    FIRST_VALUE(ProductID) OVER (PARTITION BY CustomerID ORDER BY OrderDate
                                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS FirstProductID
FROM Sales.Orders;
```

7.16 Flag any order where the OrderStatus is different from that same customer's previous order.

SOLUTION

```
-- Compare the current row to LAG() of the same column to detect a change
SELECT
    CustomerID, OrderID, OrderDate, OrderStatus,
    LAG(OrderStatus) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS PreviousStatus,
    CASE
        WHEN OrderStatus <> LAG(OrderStatus) OVER (PARTITION BY CustomerID ORDER BY OrderDate)
        THEN 1 ELSE 0
    END AS StatusChanged
FROM Sales.Orders;
```

Data Cleaning

8.1 Replace any NULL customer Score with 0 for reporting purposes.

SOLUTION

```
-- COALESCE returns the first non-NULL argument in the list
SELECT CustomerID, COALESCE(Score, 0) AS Score
FROM Sales.Customers;
```

8.2 Safely calculate Sales / Quantity per order, avoiding a divide-by-zero error.

SOLUTION

```
-- NULLIF turns a 0 into NULL so the division returns NULL instead of erroring
SELECT OrderID, Sales, Quantity,
       Sales / NULLIF(Quantity, 0) AS PricePerUnit
FROM Sales.Orders;
```

8.3 Return every customer's first and last name trimmed of whitespace and in uppercase.

SOLUTION

```
SELECT
    UPPER(LTRIM(RTRIM(FirstName))) AS FirstName,
    UPPER(LTRIM(RTRIM(LastName)))  AS LastName
FROM Sales.Customers;
```

8.4 Build a single FullName column by concatenating FirstName and LastName.

SOLUTION

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName
FROM Sales.Customers;
```

8.5 Return the first 3 characters of each customer's Country value.

SOLUTION

```
SELECT Country, SUBSTRING(Country, 1, 3) AS CountryCode
FROM Sales.Customers;
```

8.6 Replace every space in ShipAddress with an underscore.

SOLUTION

```
SELECT OrderID, ShipAddress, REPLACE(ShipAddress, ' ', '_') AS ShipAddressClean
FROM Sales.Orders;
```

8.7 Find orders whose ShipAddress is suspiciously short (fewer than 5 characters).

SOLUTION

```
-- LEN() returns the character length of a string, ignoring trailing spaces
SELECT OrderID, ShipAddress, LEN(ShipAddress) AS AddressLength
FROM Sales.Orders
WHERE LEN(ShipAddress) < 5;
```

8.8 Extract the year, month and day parts of OrderDate into separate columns.

SOLUTION

```
SELECT OrderID, OrderDate,
       YEAR(OrderDate) AS OrderYear,
       MONTH(OrderDate) AS OrderMonth,
       DAY(OrderDate) AS OrderDay
FROM Sales.Orders;
```

8.9 Calculate how many days it took to ship each order (ShipDate minus OrderDate).

SOLUTION

```
-- DATEDIFF(unit, start, end) returns a signed integer difference
SELECT OrderID, OrderDate, ShipDate,
       DATEDIFF(DAY, OrderDate, ShipDate) AS ShippingDays
FROM Sales.Orders;
```

8.10 Calculate the expected delivery date as 7 days after OrderDate.

SOLUTION

```
-- DATEADD(unit, number, date) shifts a date forward or backward
SELECT OrderID, OrderDate,
       DATEADD(DAY, 7, OrderDate) AS ExpectedDeliveryDate
FROM Sales.Orders;
```

8.11 Format OrderDate as a 'yyyy-MM' string for monthly reporting.

SOLUTION

```
SELECT OrderID, OrderDate, FORMAT(OrderDate, 'yyyy-MM') AS OrderMonthKey
FROM Sales.Orders;
```

8.12 Find orders where ShipDate is earlier than OrderDate (a data quality problem).

SOLUTION

```
SELECT OrderID, OrderDate, ShipDate
FROM Sales.Orders
WHERE ShipDate < OrderDate;
```

8.13 Standardize inconsistent country spellings ('USA', 'US' -> 'United States') using CASE.

SOLUTION

```
SELECT
    CustomerID,
    CASE
        WHEN Country IN ('USA', 'US') THEN 'United States'
        WHEN Country = 'UK'           THEN 'United Kingdom'
        ELSE Country
    END AS CountryStandardized
FROM Sales.Customers;
```

DDL — Database Structure

9.1 Create a new Sales.Regions table with a surrogate key and a region name.

SOLUTION

```
CREATE TABLE Sales.Regions (
  RegionID INT IDENTITY(1,1) PRIMARY KEY, -- auto-incrementing surrogate key
  RegionName VARCHAR(50) NOT NULL
);
```

9.2 Add a new Email column to the Customers table.

SOLUTION

```
ALTER TABLE Sales.Customers
ADD Email VARCHAR(100) NULL;
```

9.3 Add a CHECK constraint that prevents Products.Price from ever being negative.

SOLUTION

```
ALTER TABLE Sales.Products
ADD CONSTRAINT CK_Products_Price CHECK (Price >= 0);
```

9.4 Create an OrderDetails table with a composite relationship to Orders and Products (PK + FK).

SOLUTION

```
CREATE TABLE Sales.OrderDetails (
  OrderDetailID INT IDENTITY(1,1) PRIMARY KEY,
  OrderID INT NOT NULL,
  ProductID INT NOT NULL,
  LineQuantity INT NOT NULL,
  CONSTRAINT FK_OrderDetails_Orders FOREIGN KEY (OrderID) REFERENCES Sales.Orders(OrderID),
  CONSTRAINT FK_OrderDetails_Products FOREIGN KEY (ProductID) REFERENCES Sales.Products(ProductID)
);
```

9.5 Give OrderStatus a DEFAULT value of 'Pending' for any future insert that omits it.

SOLUTION

```
ALTER TABLE Sales.Orders
ADD CONSTRAINT DF_Orders_OrderStatus DEFAULT 'Pending' FOR OrderStatus;
```

9.6 Remove the Email column that was added earlier to Customers.

SOLUTION

```
ALTER TABLE Sales.Customers
DROP COLUMN Email;
```

9.7 Create a dedicated Staging schema to hold raw ETL landing tables.

SOLUTION

```
CREATE SCHEMA Staging;
```

9.8 Empty a staging table completely and instantly before the next ETL load.

SOLUTION

```
-- TRUNCATE is faster than DELETE because it does not log individual row deletes
TRUNCATE TABLE Staging.OrdersRaw;
```

DML — Data Manipulation

10.1 Insert a new customer record.

SOLUTION

```
INSERT INTO Sales.Customers (CustomerID, FirstName, LastName, Country, Score)
VALUES (9001, 'Youssef', 'Ahmed', 'Egypt', 450);
```

10.2 Insert three new products in a single statement.

SOLUTION

```
INSERT INTO Sales.Products (ProductID, Product, Category, Price)
VALUES
    (501, 'Trail Helmet', 'Accessories', 45),
    (502, 'Carbon Frame', 'Bikes', 899),
    (503, 'Bike Lock', 'Accessories', 25);
```

10.3 Update a specific customer's Score by CustomerID.

SOLUTION

```
UPDATE Sales.Customers
SET Score = 700
WHERE CustomerID = 9001;
```

10.4 Increase the price of every product in the 'Bikes' category by 10%.

SOLUTION

```
UPDATE Sales.Products
SET Price = Price * 1.10
WHERE Category = 'Bikes';
```

10.5 Delete archived orders older than 5 years from OrdersArchive.

SOLUTION

```
DELETE FROM Sales.OrdersArchive
WHERE OrderDate < DATEADD(YEAR, -5, GETDATE());
```

10.6 Move all orders with status 'Delivered' from Orders into OrdersArchive.

SOLUTION

```
-- INSERT INTO ... SELECT copies rows from one table straight into another
INSERT INTO Sales.OrdersArchive
    (OrderID, ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate,
     OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime)
SELECT
    OrderID, ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate,
    OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime
FROM Sales.Orders
WHERE OrderStatus = 'Delivered';
```

10.7 Update every employee's Department to its full name using a lookup join (e.g. 'HR' -> 'Human Resources').

SOLUTION

```
-- UPDATE ... FROM lets an UPDATE reference a second table via a join
UPDATE e
SET e.Department = d.FullName
FROM Sales.Employees AS e
INNER JOIN (VALUES ('HR', 'Human Resources'), ('IT', 'Information Technology'))
    AS d(ShortName, FullName)
ON e.Department = d.ShortName;
```

10.8 Remove duplicate OrdersArchive rows (same OrderID appearing more than once), keeping the newest.

SOLUTION

```
-- CTE + ROW_NUMBER makes the CTE itself deletable
WITH Ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY OrderID ORDER BY CreationTime DESC) AS rn
    FROM Sales.OrdersArchive
)
DELETE FROM Ranked WHERE rn > 1;
```

Upsert / MERGE

11.1 Use MERGE to insert new customers from a staging table and update existing ones.

SOLUTION

```
-- MERGE compares source to target in one pass and branches on match/no-match
MERGE Sales.Customers AS target
USING Staging.CustomersStaging AS source
  ON target.CustomerID = source.CustomerID
WHEN MATCHED THEN
  UPDATE SET target.FirstName = source.FirstName,
             target.LastName = source.LastName,
             target.Country  = source.Country,
             target.Score    = source.Score
WHEN NOT MATCHED BY TARGET THEN
  INSERT (CustomerID, FirstName, LastName, Country, Score)
  VALUES (source.CustomerID, source.FirstName, source.LastName, source.Country, source.Score);
```

11.2 Use MERGE to refresh product prices from a supplier price-feed staging table.

SOLUTION

```
MERGE Sales.Products AS target
USING Staging.ProductPriceFeed AS source
  ON target.ProductID = source.ProductID
WHEN MATCHED AND target.Price <> source.NewPrice THEN
  UPDATE SET target.Price = source.NewPrice
WHEN NOT MATCHED BY TARGET THEN
  INSERT (ProductID, Product, Category, Price)
  VALUES (source.ProductID, source.Product, source.Category, source.NewPrice);
```

11.3 Use MERGE to apply a batch of order-status updates coming from staging.

SOLUTION

```
MERGE Sales.Orders AS target
USING Staging.OrderStatusUpdates AS source
  ON target.OrderID = source.OrderID
WHEN MATCHED THEN
  UPDATE SET target.OrderStatus = source.NewStatus,
             target.ShipDate   = source.NewShipDate;
```

11.4 Use MERGE to delete staging rows that no longer exist in the live Orders table.

SOLUTION

```
-- WHEN NOT MATCHED BY SOURCE branches on rows only the TARGET has
MERGE Staging.OrdersMirror AS target
USING Sales.Orders AS source
  ON target.OrderID = source.OrderID
WHEN NOT MATCHED BY SOURCE THEN
  DELETE;
```

11.5 Use MERGE with an OUTPUT clause to log every insert/update it performs.

SOLUTION

```
MERGE Sales.Customers AS target
USING Staging.CustomersStaging AS source
  ON target.CustomerID = source.CustomerID
WHEN MATCHED THEN
  UPDATE SET target.Score = source.Score
WHEN NOT MATCHED BY TARGET THEN
  INSERT (CustomerID, FirstName, LastName, Country, Score)
  VALUES (source.CustomerID, source.FirstName, source.LastName, source.Country, source.Score)
  OUTPUT $action, inserted.CustomerID, inserted.Score;
```


Transactions

12.1 Wrap two related UPDATE statements in a single transaction so they succeed or fail together.

SOLUTION

```
BEGIN TRANSACTION;

UPDATE Sales.Customers SET Score = Score + 10 WHERE CustomerID = 9001;
UPDATE Sales.Orders SET OrderStatus = 'Reviewed' WHERE CustomerID = 9001;

COMMIT TRANSACTION;
```

12.2 Use TRY/CATCH to roll back a transaction automatically if any statement inside it fails.

SOLUTION

```
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Sales.Products SET Price = Price - 10 WHERE ProductID = 501;
    UPDATE Sales.Products SET Price = Price + 10 WHERE ProductID = 999999; -- does not exist

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
```

12.3 Move an order into OrdersArchive and delete it from Orders atomically.

SOLUTION

```
BEGIN TRANSACTION;

INSERT INTO Sales.OrdersArchive
    (OrderID, ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate,
     OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime)
SELECT OrderID, ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate,
    OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime
FROM Sales.Orders WHERE OrderID = 1001;

DELETE FROM Sales.Orders WHERE OrderID = 1001;

COMMIT TRANSACTION;
```

12.4 Use a SAVE TRANSACTION point to roll back only part of a larger transaction.

SOLUTION

```
BEGIN TRANSACTION;

UPDATE Sales.Customers SET Score = 100 WHERE CustomerID = 9001;
SAVE TRANSACTION BeforeRiskyUpdate;

UPDATE Sales.Customers SET Score = -50 WHERE CustomerID = 9001; -- invalid business value
ROLLBACK TRANSACTION BeforeRiskyUpdate; -- undo only the risky update

COMMIT TRANSACTION;
```

12.5 Run a read using an explicit isolation level so it does not block concurrent writers.

SOLUTION

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

SELECT * FROM Sales.Orders WHERE OrderStatus = 'Pending';
```

Temporary Tables

13.1 Create a session-scoped temp table holding only high-value customers (Score > 700).

SOLUTION

```
-- A single '#' prefix creates a local temp table visible only in this session
SELECT CustomerID, FirstName, LastName, Score
INTO #TempHighValueCustomers
FROM Sales.Customers
WHERE Score > 700;

SELECT * FROM #TempHighValueCustomers;
```

13.2 Use a table variable to hold this month's top 5 products by sales.

SOLUTION

```
-- Table variables (@name) have a narrower scope than temp tables (batch-level)
DECLARE @TopProducts TABLE (ProductID INT, TotalSales INT);

INSERT INTO @TopProducts (ProductID, TotalSales)
SELECT TOP (5) ProductID, SUM(Sales)
FROM Sales.Orders
GROUP BY ProductID
ORDER BY SUM(Sales) DESC;

SELECT * FROM @TopProducts;
```

13.3 Stage a join result in a temp table, then run a second, simpler query against it.

SOLUTION

```
SELECT o.OrderID, c.FirstName, c.Country, o.Sales
INTO #OrderCustomer
FROM Sales.Orders AS o
INNER JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID;

SELECT Country, SUM(Sales) AS TotalSales
FROM #OrderCustomer
GROUP BY Country;
```

13.4 Stage cleaned records in a temp table before running a MERGE into the target table.

SOLUTION

```
SELECT DISTINCT CustomerID, LTRIM(RTRIM(FirstName)) AS FirstName,
                    LTRIM(RTRIM(LastName)) AS LastName, Country, Score
INTO #StagedCustomers
FROM Staging.CustomersRaw;

MERGE Sales.Customers AS target
USING #StagedCustomers AS source
    ON target.CustomerID = source.CustomerID
WHEN MATCHED THEN UPDATE SET target.Score = source.Score
WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, FirstName, LastName, Country, Score)
    VALUES (source.CustomerID, source.FirstName, source.LastName, source.Country, source.Score);

DROP TABLE #StagedCustomers;
```

Views

14.1 Create a view that shows every customer next to their total order value.

SOLUTION

```
CREATE VIEW Sales.vw_CustomerOrderTotals AS
SELECT
    c.CustomerID, c.FirstName, c.LastName,
    COALESCE(SUM(o.Sales), 0) AS TotalSales
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName;
```

14.2 Create a view summarizing total quantity and sales per product.

SOLUTION

```
CREATE VIEW Sales.vw_ProductSales AS
SELECT
    p.ProductID, p.Product, p.Category,
    SUM(o.Quantity) AS TotalQuantity,
    SUM(o.Sales) AS TotalSales
FROM Sales.Products AS p
INNER JOIN Sales.Orders AS o ON p.ProductID = o.ProductID
GROUP BY p.ProductID, p.Product, p.Category;
```

14.3 Modify the customer-order-totals view to also expose the customer's Country.

SOLUTION

```
ALTER VIEW Sales.vw_CustomerOrderTotals AS
SELECT
    c.CustomerID, c.FirstName, c.LastName, c.Country,
    COALESCE(SUM(o.Sales), 0) AS TotalSales
FROM Sales.Customers AS c
LEFT JOIN Sales.Orders AS o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName, c.Country;
```

14.4 Create a view that always shows each customer's single latest order status.

SOLUTION

```
CREATE VIEW Sales.vw_LatestOrderStatus AS
SELECT CustomerID, OrderID, OrderStatus, OrderDate
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC) AS rn
    FROM Sales.Orders
) AS ranked
WHERE rn = 1;
```

14.5 Remove the product sales view because it is no longer needed.

SOLUTION

```
DROP VIEW Sales.vw_ProductSales;
```

Stored Procedures

15.1 Create a procedure that returns all orders for a given CustomerID.

SOLUTION

```
CREATE PROCEDURE Sales.usp_GetOrdersByCustomer
    @CustomerID INT
AS
BEGIN
    SELECT OrderID, ProductID, OrderDate, Sales
    FROM Sales.Orders
    WHERE CustomerID = @CustomerID;
END;
```

15.2 Create a procedure that inserts a customer only if that CustomerID does not already exist.

SOLUTION

```
CREATE PROCEDURE Sales.usp_AddCustomerIfNew
    @CustomerID INT, @FirstName VARCHAR(50), @LastName VARCHAR(50), @Country VARCHAR(50)
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Sales.Customers WHERE CustomerID = @CustomerID)
    BEGIN
        INSERT INTO Sales.Customers (CustomerID, FirstName, LastName, Country)
        VALUES (@CustomerID, @FirstName, @LastName, @Country);
    END
    ELSE
    BEGIN
        PRINT 'Customer already exists – no insert performed.';
    END
END;
```

15.3 Create a procedure that returns a customer's total sales through an OUTPUT parameter.

SOLUTION

```
CREATE PROCEDURE Sales.usp_GetCustomerTotalSales
    @CustomerID INT,
    @TotalSales INT OUTPUT
AS
BEGIN
    SELECT @TotalSales = COALESCE(SUM(Sales), 0)
    FROM Sales.Orders
    WHERE CustomerID = @CustomerID;
END;
```

15.4 Create a procedure that safely updates a product's price and reports any error.

SOLUTION

```
CREATE PROCEDURE Sales.usp_UpdateProductPrice
    @ProductID INT, @NewPrice INT
AS
BEGIN
    BEGIN TRY
        UPDATE Sales.Products SET Price = @NewPrice WHERE ProductID = @ProductID;
    END TRY
    BEGIN CATCH
        PRINT 'Error updating price: ' + ERROR_MESSAGE();
    END CATCH
END;
```

15.5 Create a procedure that processes archival in yearly batches using a WHILE loop.

SOLUTION

```
CREATE PROCEDURE Sales.usp_ArchiveOldOrdersByYear
    @StartYear INT, @EndYear INT
AS
BEGIN
    DECLARE @CurrentYear INT = @StartYear;

    WHILE @CurrentYear <= @EndYear
    BEGIN
        INSERT INTO Sales.OrdersArchive
            SELECT * FROM Sales.Orders WHERE YEAR(OrderDate) = @CurrentYear;

        DELETE FROM Sales.Orders WHERE YEAR(OrderDate) = @CurrentYear;

        SET @CurrentYear = @CurrentYear + 1; -- advance to the next year
    END
END;
```

15.6 Create a procedure that performs an incremental upsert of new/changed orders from staging.

SOLUTION

```
CREATE PROCEDURE Sales.usp_LoadOrdersIncremental
AS
BEGIN
    MERGE Sales.Orders AS target
    USING Staging.OrdersIncoming AS source
        ON target.OrderID = source.OrderID
    WHEN MATCHED THEN
        UPDATE SET target.OrderStatus = source.OrderStatus,
                   target.ShipDate = source.ShipDate
    WHEN NOT MATCHED BY TARGET THEN
        INSERT (OrderID, ProductID, CustomerID, SalesPersonID, OrderDate,
                ShipDate, OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime)
        VALUES (source.OrderID, source.ProductID, source.CustomerID, source.SalesPersonID,
                source.OrderDate, source.ShipDate, source.OrderStatus, source.ShipAddress,
                source.BillAddress, source.Quantity, source.Sales, source.CreationTime);
END;
```

Indexes

16.1 Create an index to speed up lookups of orders by CustomerID.

SOLUTION

```
CREATE INDEX IX_Orders_CustomerID
ON Sales.Orders (CustomerID);
```

16.2 Create an index to speed up date-range filters on Orders.

SOLUTION

```
CREATE INDEX IX_Orders_OrderDate
ON Sales.Orders (OrderDate);
```

16.3 Create a unique index guaranteeing no two products share the same name.

SOLUTION

```
CREATE UNIQUE INDEX UX_Products_ProductName
ON Sales.Products (Product);
```

16.4 Create a composite index to speed up 'orders for a customer within a date range' queries.

SOLUTION

```
-- Column order matters: put the equality column (CustomerID) first
CREATE INDEX IX_Orders_Customer_Date
ON Sales.Orders (CustomerID, OrderDate);
```

16.5 Remove the OrderDate index because it is no longer used.

SOLUTION

```
DROP INDEX IX_Orders_OrderDate ON Sales.Orders;
```

16.6 Create a covering index so a common report query never has to touch the base table.

SOLUTION

```
-- INCLUDE adds non-key columns to the index leaf level for 'covering' reads
CREATE INDEX IX_Orders_Covering
ON Sales.Orders (CustomerID)
INCLUDE (OrderDate, Sales, OrderStatus);
```

Query Performance

17.1 Measure the logical reads and execution time of a query.

SOLUTION

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT CustomerID, SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY CustomerID;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

17.2 Rewrite a wasteful SELECT * report query to only pull the columns it actually needs.

SOLUTION

```
-- Slow / wasteful version:
-- SELECT * FROM Sales.Orders WHERE CustomerID = 9001;

-- Faster version: only the columns the report actually displays
SELECT OrderID, OrderDate, Sales
FROM Sales.Orders
WHERE CustomerID = 9001;
```

17.3 Filter rows as early as possible before joining, instead of joining everything and filtering after.

SOLUTION

```
-- Filtering inside a derived table shrinks the row set BEFORE the join runs
SELECT o.OrderID, c.FirstName
FROM (SELECT * FROM Sales.Orders WHERE OrderStatus = 'Pending') AS o
INNER JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID;
```

17.4 Rewrite a sargable-breaking date filter so it can use an index.

SOLUTION

```
-- Non-sargable (wraps the column in a function, index cannot be used):
-- WHERE YEAR(OrderDate) = 2026

-- Sargable rewrite: a plain range comparison can seek an index on OrderDate
SELECT * FROM Sales.Orders
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01';
```

17.5 Check for existence of at least one 'Cancelled' order using EXISTS instead of COUNT(*).

SOLUTION

```
-- EXISTS can stop at the first match; COUNT(*) forces a full scan of matches
IF EXISTS (SELECT 1 FROM Sales.Orders WHERE OrderStatus = 'Cancelled')
    PRINT 'At least one cancelled order found.';
```

Data Warehouse Concepts

18.1 Verify the grain of the Orders table by checking that OrderID is unique (one row per order).

SOLUTION

```
-- If this returns 0 rows, the grain really is 'one row per OrderID'
SELECT OrderID, COUNT(*) AS Cnt
FROM Sales.Orders
GROUP BY OrderID
HAVING COUNT(*) > 1;
```

18.2 Aggregate the fact table's core measures (Quantity, Sales) into a single summary row.

SOLUTION

```
-- Quantity and Sales are the 'measures' of this fact table
SELECT SUM(Quantity) AS TotalQuantity, SUM(Sales) AS TotalSales
FROM Sales.Orders;
```

18.3 Join the Orders fact table to the Products dimension to enrich each fact row with descriptive attributes.

SOLUTION

```
SELECT o.OrderID, o.Sales, p.Product, p.Category
FROM Sales.Orders AS o
INNER JOIN Sales.Products AS p ON o.ProductID = p.ProductID;
```

18.4 Demonstrate the difference between an additive measure (Sales) and a non-additive one (Price) when summed by category.

SOLUTION

```
-- Summing Sales across orders is meaningful; summing Price is not (non-additive)
SELECT p.Category,
       SUM(o.Sales) AS TotalSales,      -- additive: valid to sum
       SUM(p.Price) AS SumOfPrices     -- non-additive: not a meaningful business number
FROM Sales.Orders AS o
INNER JOIN Sales.Products AS p ON o.ProductID = p.ProductID
GROUP BY p.Category;
```

18.5 Produce a customer dimension extract keyed by the natural/business key CustomerID.

SOLUTION

```
SELECT CustomerID AS CustomerBusinessKey, FirstName, LastName, Country
FROM Sales.Customers;
```


Dimensional Modeling

19.1 Create the skeleton of a star schema: FactSales plus DimCustomer, DimProduct and DimDate.

SOLUTION

```
CREATE TABLE dbo.DimCustomer (customer_key INT IDENTITY PRIMARY KEY, CustomerID INT, FirstName VARCHAR(50), Country
VARCHAR(50));
CREATE TABLE dbo.DimProduct (product_key INT IDENTITY PRIMARY KEY, ProductID INT, Product VARCHAR(50), Category VARCHAR(50));
CREATE TABLE dbo.DimDate (date_key INT PRIMARY KEY, FullDate DATE, Year INT, Month INT, Day INT);

CREATE TABLE dbo.FactSales (
    customer_key INT REFERENCES dbo.DimCustomer(customer_key),
    product_key INT REFERENCES dbo.DimProduct(product_key),
    date_key INT REFERENCES dbo.DimDate(date_key),
    Quantity INT,
    Sales INT
);
```

19.2 Query FactSales joined to all three dimensions to report sales by customer, product and date (a star-schema query).

SOLUTION

```
SELECT c.FirstName, p.Product, d.FullDate, f.Sales
FROM dbo.FactSales AS f
INNER JOIN dbo.DimCustomer AS c ON f.customer_key = c.customer_key
INNER JOIN dbo.DimProduct AS p ON f.product_key = p.product_key
INNER JOIN dbo.DimDate AS d ON f.date_key = d.date_key;
```

19.3 Normalize DimProduct into a snowflake by splitting Category out into its own DimCategory table.

SOLUTION

```
CREATE TABLE dbo.DimCategory (category_key INT IDENTITY PRIMARY KEY, CategoryName VARCHAR(50));
ALTER TABLE dbo.DimProduct ADD category_key INT REFERENCES dbo.DimCategory(category_key);

-- Snowflake query: one extra hop compared to the flat star-schema version
SELECT p.Product, cat.CategoryName
FROM dbo.DimProduct AS p
INNER JOIN dbo.DimCategory AS cat ON p.category_key = cat.category_key;
```

19.4 Populate DimDate with one row per calendar day for the year 2026 using a recursive CTE.

SOLUTION

```
WITH DateSeq AS (
    SELECT CAST('2026-01-01' AS DATE) AS FullDate
    UNION ALL
    SELECT DATEADD(DAY, 1, FullDate) FROM DateSeq WHERE FullDate < '2026-12-31'
)
INSERT INTO dbo.DimDate (date_key, FullDate, Year, Month, Day)
SELECT CONVERT(INT, FORMAT(FullDate, 'yyyyMMdd')), FullDate,
    YEAR(FullDate), MONTH(FullDate), DAY(FullDate)
FROM DateSeq
OPTION (MAXRECURSION 366);
```

19.5 Report total sales by year and category using the star schema.

SOLUTION

```
SELECT d.Year, p.Category, SUM(f.Sales) AS TotalSales
FROM dbo.FactSales AS f
INNER JOIN dbo.DimDate AS d ON f.date_key = d.date_key
INNER JOIN dbo.DimProduct AS p ON f.product_key = p.product_key
GROUP BY d.Year, p.Category;
```

Surrogate Keys

20.1 Add an IDENTITY surrogate key column to DimCustomer.

SOLUTION

```
ALTER TABLE dbo.DimCustomer
ADD customer_key INT IDENTITY(1,1);
```

20.2 Look up a customer's surrogate key from their business key (CustomerID).

SOLUTION

```
SELECT customer_key, CustomerID
FROM dbo.DimCustomer
WHERE CustomerID = 9001;
```

20.3 Insert new fact rows into FactSales, translating natural keys into surrogate keys via a join.

SOLUTION

```
INSERT INTO dbo.FactSales (customer_key, product_key, date_key, Quantity, Sales)
SELECT
    dc.customer_key,
    dp.product_key,
    CONVERT(INT, FORMAT(o.OrderDate, 'yyyyMMdd')),
    o.Quantity,
    o.Sales
FROM Sales.Orders AS o
INNER JOIN dbo.DimCustomer AS dc ON o.CustomerID = dc.CustomerID
INNER JOIN dbo.DimProduct AS dp ON o.ProductID = dp.ProductID;
```

20.4 Show a dimension row with both its business key and its surrogate key side by side.

SOLUTION

```
-- Business key (CustomerID) stays stable in the source; surrogate key is internal to the DW
SELECT customer_key AS SurrogateKey, CustomerID AS BusinessKey, FirstName
FROM dbo.DimCustomer;
```

Slowly Changing Dimensions (SCD)

21.1 Create a Type 2 customer dimension table with history-tracking columns.

SOLUTION

```
CREATE TABLE dbo.DimCustomer_SCD2 (
    customer_key INT IDENTITY PRIMARY KEY,
    CustomerID    INT NOT NULL,        -- business key
    Country       VARCHAR(50),
    StartDate     DATE NOT NULL,
    EndDate       DATE NULL,
    IsCurrent      BIT NOT NULL
);
```

21.2 Perform the initial load of DimCustomer_SCD2, marking every row as the current version.

SOLUTION

```
INSERT INTO dbo.DimCustomer_SCD2 (CustomerID, Country, StartDate, EndDate, IsCurrent)
SELECT CustomerID, Country, CAST(GETDATE() AS DATE), NULL, 1
FROM Sales.Customers;
```

21.3 Expire the current record for a customer whose Country has changed (Type 2, step 1 of 2).

SOLUTION

```
UPDATE dbo.DimCustomer_SCD2
SET EndDate = CAST(GETDATE() AS DATE), IsCurrent = 0
WHERE CustomerID = 9001 AND IsCurrent = 1;
```

21.4 Insert the new version of that customer's record with the updated Country (Type 2, step 2 of 2).

SOLUTION

```
INSERT INTO dbo.DimCustomer_SCD2 (CustomerID, Country, StartDate, EndDate, IsCurrent)
VALUES (9001, 'United Arab Emirates', CAST(GETDATE() AS DATE), NULL, 1);
```

21.5 Query only the current version of every customer.

SOLUTION

```
SELECT * FROM dbo.DimCustomer_SCD2 WHERE IsCurrent = 1;
```

21.6 Query the full change history for one specific customer.

SOLUTION

```
SELECT customer_key, Country, StartDate, EndDate, IsCurrent
FROM dbo.DimCustomer_SCD2
WHERE CustomerID = 9001
ORDER BY StartDate;
```

21.7 Apply a Type 1 update to a customer's Score: overwrite in place, keeping no history.

SOLUTION

```
-- SCD Type 1: no new row, no StartDate/EndDate — the old value is simply gone
UPDATE Sales.Customers
SET Score = 820
WHERE CustomerID = 9001;
```

21.8 Implement the whole Type 2 expire-and-insert pattern in a single MERGE statement.

SOLUTION

```
MERGE dbo.DimCustomer_SCD2 AS target
USING Staging.CustomersIncoming AS source
    ON target.CustomerID = source.CustomerID AND target.IsCurrent = 1
WHEN MATCHED AND target.Country <> source.Country THEN
    UPDATE SET target.EndDate = CAST(GETDATE() AS DATE), target.IsCurrent = 0
WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, Country, StartDate, EndDate, IsCurrent)
    VALUES (source.CustomerID, source.Country, CAST(GETDATE() AS DATE), NULL, 1);
-- Note: a second INSERT pass is still needed to add the new current row
-- for customers whose Country changed, since MERGE cannot UPDATE and
-- INSERT the same logical customer in one WHEN clause.
```

ETL — Extract

22.1 Perform a full extraction of every row from the Orders source table.

SOLUTION

```
-- Full extraction: reads the entire table every run (simple, but not scalable)
SELECT * FROM Sales.Orders;
```

22.2 Perform an incremental extraction using a CreationTime watermark.

SOLUTION

```
-- Only rows created after the last successful run are extracted
DECLARE @LastWatermark DATETIME2 = '2026-08-01 00:00:00';

SELECT * FROM Sales.Orders
WHERE CreationTime > @LastWatermark;
```

22.3 Perform an ID-based incremental extraction using the last processed OrderID.

SOLUTION

```
DECLARE @LastProcessedID INT = 5000;

SELECT * FROM Sales.Orders
WHERE OrderID > @LastProcessedID;
```

22.4 Extract only employee records changed since the last run (assumes an UpdatedAt column exists).

SOLUTION

```
-- Requires an UpdatedAt column maintained by the source system or a trigger
DECLARE @LastRun DATETIME2 = '2026-08-01 00:00:00';

SELECT * FROM Sales.Employees
WHERE UpdatedAt > @LastRun;
```

22.5 Extract a complete historical dataset by combining current and archived orders.

SOLUTION

```
SELECT * FROM Sales.Orders
UNION ALL
SELECT * FROM Sales.OrdersArchive;
```

ETL — Transform

23.1 Clean a staging copy of Orders: trim text fields, uppercase status, and coalesce NULL Sales to 0.

SOLUTION

```
SELECT
    OrderID,
    CustomerID,
    UPPER(LTRIM(RTRIM(OrderStatus))) AS OrderStatus,
    LTRIM(RTRIM(ShipAddress))      AS ShipAddress,
    COALESCE(Sales, 0)             AS Sales
FROM Staging.OrdersRaw;
```

23.2 Standardize inconsistent Gender codes ('M'/'F') into full text values.

SOLUTION

```
SELECT EmployeeID,
       CASE Gender WHEN 'M' THEN 'Male' WHEN 'F' THEN 'Female' ELSE 'Unknown' END AS GenderFull
FROM Sales.Employees;
```

23.3 Add derived OrderMonth and DeliveryDays columns during transformation.

SOLUTION

```
SELECT
    OrderID,
    FORMAT(OrderDate, 'yyyy-MM')      AS OrderMonth,
    DATEDIFF(DAY, OrderDate, ShipDate) AS DeliveryDays
FROM Sales.Orders;
```

23.4 Deduplicate incoming staged orders, keeping only the most recently loaded copy of each OrderID.

SOLUTION

```
WITH Deduped AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY OrderID ORDER BY CreationTime DESC) AS rn
    FROM Staging.OrdersRaw
)
SELECT * FROM Deduped WHERE rn = 1;
```

23.5 Apply a business rule flagging any order over 500 in Sales as 'High Value'.

SOLUTION

```
SELECT OrderID, Sales,
       CASE WHEN Sales > 500 THEN 1 ELSE 0 END AS IsHighValueOrder
FROM Sales.Orders;
```

23.6 Enrich Orders with Customer and Product attributes into one wide silver-layer table.

SOLUTION

```
-- SELECT INTO creates a new physical table from the query result
SELECT
    o.OrderID, o.OrderDate, o.Sales,
    c.FirstName, c.LastName, c.Country,
    p.Product, p.Category
INTO Silver.OrdersEnriched
FROM Sales.Orders AS o
INNER JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID
INNER JOIN Sales.Products AS p ON o.ProductID = p.ProductID;
```

ETL — Load

24.1 Perform a full load into the gold fact table: clear it out, then reload everything.

SOLUTION

```
TRUNCATE TABLE Gold.FactSales;

INSERT INTO Gold.FactSales (OrderID, CustomerID, ProductID, Sales)
SELECT OrderID, CustomerID, ProductID, Sales
FROM Silver.OrdersEnriched;
```

24.2 Perform an incremental append load, inserting only OrderIDs not already in the fact table.

SOLUTION

```
INSERT INTO Gold.FactSales (OrderID, CustomerID, ProductID, Sales)
SELECT s.OrderID, s.CustomerID, s.ProductID, s.Sales
FROM Silver.OrdersEnriched AS s
WHERE NOT EXISTS (
    SELECT 1 FROM Gold.FactSales AS f WHERE f.OrderID = s.OrderID
);
```

24.3 Upsert DimCustomer using MERGE so the load can run repeatedly without creating duplicates.

SOLUTION

```
MERGE Gold.DimCustomer AS target
USING Silver.CustomersClean AS source
    ON target.CustomerID = source.CustomerID
WHEN MATCHED THEN
    UPDATE SET target.FirstName = source.FirstName, target.Country = source.Country
WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, FirstName, Country)
VALUES (source.CustomerID, source.FirstName, source.Country);
```

24.4 Load a large staging file efficiently using the bulk-insert pattern.

SOLUTION

```
-- BULK INSERT streams a flat file directly into SQL Server, bypassing row-by-row inserts
BULK INSERT Staging.OrdersRaw
FROM '/data/exports/orders_2026_08.csv'
WITH (
    FIELDTERMINATOR = ',',
    ROWTERMINATOR   = '\n',
    FIRSTROW        = 2      -- skip the header row
);
```

24.5 Overwrite a monthly summary aggregate table with freshly recomputed totals.

SOLUTION

```
TRUNCATE TABLE Gold.MonthlySalesSummary;

INSERT INTO Gold.MonthlySalesSummary (OrderMonth, TotalSales)
SELECT FORMAT(OrderDate, 'yyyy-MM'), SUM(Sales)
FROM Sales.Orders
GROUP BY FORMAT(OrderDate, 'yyyy-MM');
```

Incremental ETL

25.1 Create a watermark table to track the last successfully processed timestamp per source.

SOLUTION

```
CREATE TABLE dbo.ETL_Watermark (  
    SourceName    VARCHAR(100) PRIMARY KEY,  
    LastWatermark DATETIME2 NOT NULL  
);
```

25.2 Select only the orders created after the stored watermark for the Orders source.

SOLUTION

```
SELECT o.*  
FROM Sales.Orders AS o  
CROSS JOIN (  
    SELECT LastWatermark FROM dbo.ETL_Watermark WHERE SourceName = 'Orders'  
) AS w  
WHERE o.CreationTime > w.LastWatermark;
```

25.3 Update the watermark table after a successful incremental load.

SOLUTION

```
UPDATE dbo.ETL_Watermark  
SET LastWatermark = (SELECT MAX(CreationTime) FROM Sales.Orders)  
WHERE SourceName = 'Orders';
```

25.4 Detect changed rows in Orders using a CDC-style comparison against the last run time.

SOLUTION

```
DECLARE @LastRun DATETIME2 = (SELECT LastWatermark FROM dbo.ETL_Watermark WHERE SourceName = 'Orders');  
  
SELECT * FROM Sales.Orders  
WHERE CreationTime > @LastRun;
```

25.5 Combine an incremental extract with an upsert MERGE in one end-to-end script.

SOLUTION

```
DECLARE @LastRun DATETIME2 = (SELECT LastWatermark FROM dbo.ETL_Watermark WHERE SourceName = 'Orders');  
  
MERGE Gold.FactSales AS target  
USING (  
    SELECT * FROM Sales.Orders WHERE CreationTime > @LastRun  
) AS source  
    ON target.OrderID = source.OrderID  
WHEN MATCHED THEN  
    UPDATE SET target.Sales = source.Sales  
WHEN NOT MATCHED BY TARGET THEN  
    INSERT (OrderID, CustomerID, ProductID, Sales)  
    VALUES (source.OrderID, source.CustomerID, source.ProductID, source.Sales);  
  
UPDATE dbo.ETL_Watermark  
SET LastWatermark = (SELECT MAX(CreationTime) FROM Sales.Orders)  
WHERE SourceName = 'Orders';
```


Data Quality

26.1 Check for NULLs in the critical CustomerID and ProductID columns of Orders.

SOLUTION

```
SELECT COUNT(*) AS RowsWithMissingKeys
FROM Sales.Orders
WHERE CustomerID IS NULL OR ProductID IS NULL;
```

26.2 Check for duplicate OrderID values, which should be unique.

SOLUTION

```
SELECT OrderID, COUNT(*) AS Occurrences
FROM Sales.Orders
GROUP BY OrderID
HAVING COUNT(*) > 1;
```

26.3 Check for orders with an invalid, non-positive Quantity.

SOLUTION

```
SELECT OrderID, Quantity
FROM Sales.Orders
WHERE Quantity <= 0;
```

26.4 Check for orders where ShipDate happens before OrderDate.

SOLUTION

```
SELECT OrderID, OrderDate, ShipDate
FROM Sales.Orders
WHERE ShipDate < OrderDate;
```

26.5 Check for orders referencing a ProductID that does not exist in Products (broken foreign key).

SOLUTION

```
SELECT o.OrderID, o.ProductID
FROM Sales.Orders AS o
LEFT JOIN Sales.Products AS p ON o.ProductID = p.ProductID
WHERE p.ProductID IS NULL;
```

26.6 Check for orders referencing a CustomerID that does not exist in Customers.

SOLUTION

```
SELECT o.OrderID, o.CustomerID
FROM Sales.Orders AS o
LEFT JOIN Sales.Customers AS c ON o.CustomerID = c.CustomerID
WHERE c.CustomerID IS NULL;
```

26.7 Check for customer Score values that fall outside the expected 0-1000 range.

SOLUTION

```
SELECT CustomerID, Score
FROM Sales.Customers
WHERE Score IS NOT NULL AND (Score < 0 OR Score > 1000);
```

26.8 Produce a single combined data-quality summary report covering several checks at once.

SOLUTION

```
-- Each UNION ALL branch is one independent rule; the label identifies which rule fired
SELECT 'Missing Keys'      AS CheckName, COUNT(*) AS IssueCount FROM Sales.Orders WHERE CustomerID IS NULL OR ProductID IS NULL
UNION ALL
SELECT 'Non-Positive Qty', COUNT(*) FROM Sales.Orders WHERE Quantity <= 0
UNION ALL
SELECT 'Ship Before Order', COUNT(*) FROM Sales.Orders WHERE ShipDate < OrderDate
UNION ALL
SELECT 'Orphan ProductID', COUNT(*)
FROM Sales.Orders AS o LEFT JOIN Sales.Products AS p ON o.ProductID = p.ProductID
WHERE p.ProductID IS NULL;
```

Bronze / Silver / Gold

27.1 Create the Bronze layer: a raw, unmodified copy of the source Orders table.

SOLUTION

```
-- Bronze preserves the source data exactly as it arrived, for full traceability
SELECT * INTO Bronze.Orders FROM Sales.Orders;
```

27.2 Create the Silver layer: clean, validated, deduplicated data built from Bronze.

SOLUTION

```
WITH Deduped AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY OrderID ORDER BY CreationTime DESC) AS rn
    FROM Bronze.Orders
)
SELECT
    OrderID, CustomerID, ProductID,
    UPPER(LTRIM(RTRIM(OrderStatus))) AS OrderStatus,
    Quantity, Sales
INTO Silver.Orders
FROM Deduped
WHERE rn = 1 AND Quantity > 0 AND Sales IS NOT NULL; -- basic validation rules
```

27.3 Create the Gold layer: a business-ready FactSales table aggregated from Silver.

SOLUTION

```
SELECT
    CustomerID, ProductID,
    SUM(Quantity) AS TotalQuantity,
    SUM(Sales) AS TotalSales
INTO Gold.FactSales
FROM Silver.Orders
GROUP BY CustomerID, ProductID;
```

27.4 Create a Gold DimCustomer table sourced from a cleaned Silver customers table.

SOLUTION

```
SELECT DISTINCT CustomerID, FirstName, LastName, Country
INTO Gold.DimCustomer
FROM Silver.Customers;
```

27.5 Run the full Bronze -> Silver -> Gold pipeline end-to-end in sequence.

SOLUTION

```
-- Step 1: Bronze — raw landing zone, no transformation
SELECT * INTO Bronze.Orders FROM Sales.Orders;

-- Step 2: Silver — clean, dedupe, validate
SELECT DISTINCT OrderID, CustomerID, ProductID, Sales
INTO Silver.Orders
FROM Bronze.Orders
WHERE Sales IS NOT NULL;

-- Step 3: Gold — aggregate into a reporting-ready fact table
SELECT CustomerID, SUM(Sales) AS TotalSales
INTO Gold.FactSales
FROM Silver.Orders
GROUP BY CustomerID;
```